CENG-322

Deliverable 4

FINAL DOCUMENT LAYOUT

Please submit **a single PDF document per group**. This means that the document should be professionally organized and have a uniform style throughout. It should look as though it came from one team, not 4 separate students. Please note that instructors may choose to run your submission through TurnItIn or compare the submission with other students from other sections for academic integrity purposes.

A single **PDF document** with the following sections denoted using page numbers, headers/footers and a table of contents:

All team members must present, missing members will be assigned 0.

Must follow the sequence below:

Use proper formatting

Follow the steps as requested below, meaning order your content as below.

1. Document Title; Deliverable 4
2. Team Name: LIFE (*Light, Indoor air, Freshness, and Energy*)
3. Project Name: LIFE Environmental Solutions
4. Student's names and IDs
5. Table of Contents
6. In the pdf, write the requirement and comment on it for each item. Include the question and the answer.
7. Table below with signatures
8. Members Info and Participation. Effort was put on the project (0 – 100%), students will be marked based on their effort, i.e. if the team gets 8, and one student effort was only 50%, that student mark will be adjusted to 8 * 50% = 4

Name	Student Id	Github Id	Signature	Effort
Mohamed Ali	n01440760	MohamedAli0760	M.A.A	100%
Jonathan Akabuike	N01510573	JonathanAkaBuike0573	J.A.	100%
Farhan Habibzai	N01610299	FarhanHabibzai	F.H	25%

9. Brief description of the project, couple sentences.

Our project integrates a complete indoor environmental monitoring system designed to give users real-time insight into the conditions of their surrounding environment. Using a combination of advanced sensors. including a digital air quality sensor, a human presence sensor, and an ambient light sensor. the system continuously tracks key environmental factors such as air particle concentration, CO_2/VOC levels, room occupancy, and lighting conditions.

All sensor data is transmitted to a mobile or web application where users can view live readings, receive intelligent recommendations, and get alerts when conditions become unsafe or unhealthy. The human presence sensor provides accurate occupancy detection, helping determine how many people are in a given space, while the air quality sensor (such as the SPG30) measures CO_2 and identifies harmful air contaminants. The light sensor helps evaluate brightness levels to support energy-efficient lighting control

10. What issue your product will solve.:
Our solution addresses several critical indoor environmental challenges:

Invisible Air Hazards:
With real-time air quality monitoring, LIFE detects harmful particles and rising CO_2/VOC levels that would otherwise go unnoticed. This goes beyond traditional air quality reports by giving users precise, immediate information about what they are breathing indoors.

Occupancy-Driven Air Quality Insights:
By combining the human presence sensor with air quality data, LIFE directly correlates pollutant spikes to room occupancy. This is essential in high-traffic

indoor spaces—such as classrooms, offices, and small warehouses—where increased occupancy can quickly reduce oxygen levels and elevate CO_2 concentrations, leading to unhealthy and unproductive environments.

Lighting and Energy Efficiency:
Using the light sensor, LIFE monitors ambient brightness and supports smart energy-management decisions. This helps reduce energy waste while maintaining a comfortable visual environment for occupants.

Safety & Health Alerts:
LIFE provides real-time notifications and triggers ventilation, airflow adjustments, or evacuation recommendations when environmental conditions become unhealthy. These alerts are informed by sensor data and correlated with the number of people present in the space.

11. Compare your application with at least two existing apps in the market. Provide links and description of the two apps you selected.: We will select AirMatters and IQAir. AirMatters (https://air-matters.com/index.html) IQAir link (https://www.iqair.com/) Air Matters is to service focuses on air quality issues in the world, aiming to provide a handy and powerful tool for broadcasting realtime air quality and giving health advice for users. It helps with the air quality indoors and the control for it is what helps us breathe better. IQAir is to empower the world to breathe cleaner air. Through scientific innovation and global partnerships, IQAir continues this mission, championing cleaner, healthier environments with advanced air filtration technology and innovative air quality monitoring solutions. IQAir's enduring commitment over half a century underscores a relentless pursuit for a world where clean air is accessible to all.

12. Highlight the differences between your app and these two apps. LIFE Environmental will combine the Occupancy-Driven Air Quality and take action for the safety alerts for any indoor space. Also we will have Direct, real-time human presence sensor reading (people count). The IQAir Visual will want to have the Global/Local Outdoor Air Quality data with optional indoor sensor integration. It also has a separate indoor monitoring device which *may* have CO2 sensors, but not a dedicated presence count. With AirMatters, the air quality data is for the world and the issues that are with it. It will monitor the air quality and hope for real-time broadcasting to ensure we have health advice that is given to the users.

13. Why do you believe your app is better than these two apps? LIFE is better than AirMatters and Neptronic because it provides us with the most relevant and actionable safety information to the average user with a low-cost, highly customizable platform to utilize for your own making.

- Safety : AirMatters is great for tracking air quality, but it does not know how many people are in the room. LIFE will measure the occupancy with your Human Presence Sensor. This allows LIFE to provide an alert like, "CO2 is extremely high and 5 people are present." Immediate

ventilation: AirMatterswill show that CO2 is high, but our integrated data makes the risk assessment far more direct and urgent for consumers.

- Low Barrier to Entry: AirMatters requires buying one of compatible, high-end consumer monitors. LIFE is based on a DIY hardware platform using accessible breakout modules. It makes it significantly cheaper and customizable, ideal for students, teachers , small businesses, or specific non-standard environments
- Focus on Health vs. Automation/Data: While Neptronic focuses on energy efficiency through automation and AirMatters focuses on providing comprehensive data, LIFE's mission is immediate human safety based on dynamic interaction between air quality and human density.

14. Create a table pros and cons of the three different apps.

Application	Pros	Cons
LIFE	Counting occupant is integrated with air quality data for precise safety alerts. Low-cost, open-source DIY hardware platform for accessibility and customizability. Focuses on immediate human action in small spaces	Limited to only local monitoring, no global air quality data or forecasts. Requires basic DIY technical knowledge
AirMatters	Excellent global air quality data and forecasts. Can integrate with controls of several premium consumer air purifiers and monitors. Provides pollen data	Requires expensive, proprietary hardware for indoor monitoring. No dedicated occupancy sensor; hardly relies on CO2 for sensing
Neptronic	Fully automated control of large-scale HVAC/BMS systems. Designed for maximizing energy efficiency in commercial properties.	Extremely expensive and complex, requiring a full Building Management System. Not a solution for consumers or single-room usages.

15. **GitHub Repo link. All members must contribute to the repo.**

16. **Verify the link is working. Our Github link: (https://github.com/JonathanAkabuike0573/LIFE_EnvironmentalSolutions.git) this is the link to access the project!!**

17. **Login functionality, I will use the following credentials to test your app:**

Email: aaa@bbb.com **Password:** ***Admin101!*** **(this credential is good)**

Login credentials : admin1@gmail.com **Password: 1234567 (this is optional)**

18. Document any other login credentials I must use i.e. Admin vs. regular user. I should not need to send you an email to login to your app.

19. You are working on sprint 4.

20. Describe in detail, the work that has been completed by each team member in this sprint only.

Mohamed: I am getting in the purchase page on our app, if or when we need it. I am also trying to get unit testing for one of the Java classes. I have initially completed the About US page and hope to make it look better. I also initially completed the unit tests. I completed the purchase page.
Jonathan: I added an account management page, so users can change their names, phone numbers and email addresses. I added in the sensor readings from firebase. Moved all the screens related to our sensors to the bottom navigation.
Farhan: I changed so that the dark mode is working properly. I fix the strings so no hardcoded text is in android studio.

21. **Each member must have a minimum of 10 commits, counted starting Nov. 3 (Two-week Sprint).**

22. Marks deducted for members who are not meeting the minimum 10 commits (i.e. if only 8 commits, 20% percent deducted, …etc.). The work should be done over the sprint, and not in the last 48 hours prior to the submission.

23. **List number of commits for each member for this sprint only. Create a table like below:**

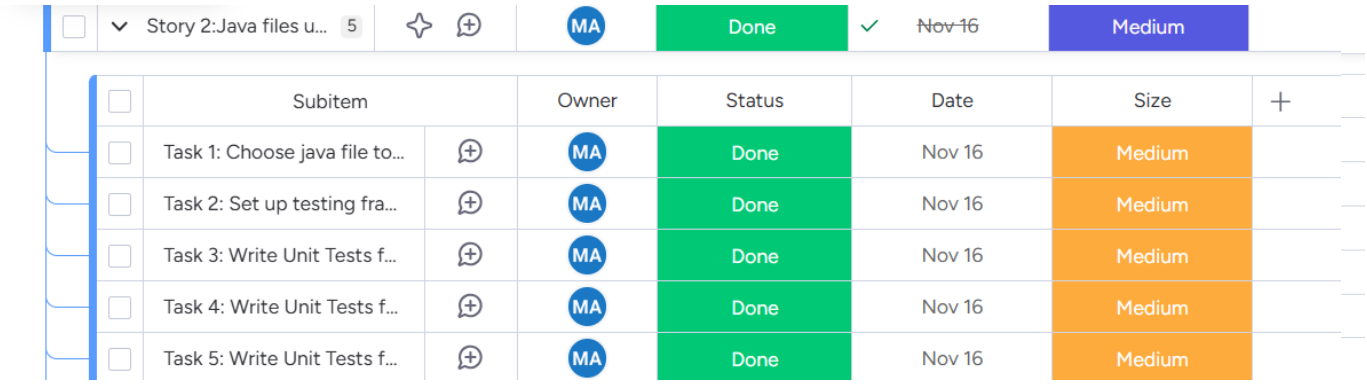
Mohamed	Jonathan	Kieran	Farhan
4	8		
10	1		
2	2		
3	1		
	3		
	2		
	4		
	10		
Total 19	31		

24. Sprint goals, list sprint goals.**: Our sprint goals are that we be able to now get our sensor readings from our sensors from our raspberry pi and be able to now put them inside our APP. We want to show what our sensor can do and to update it into our Database and be able to get it from the DB.**

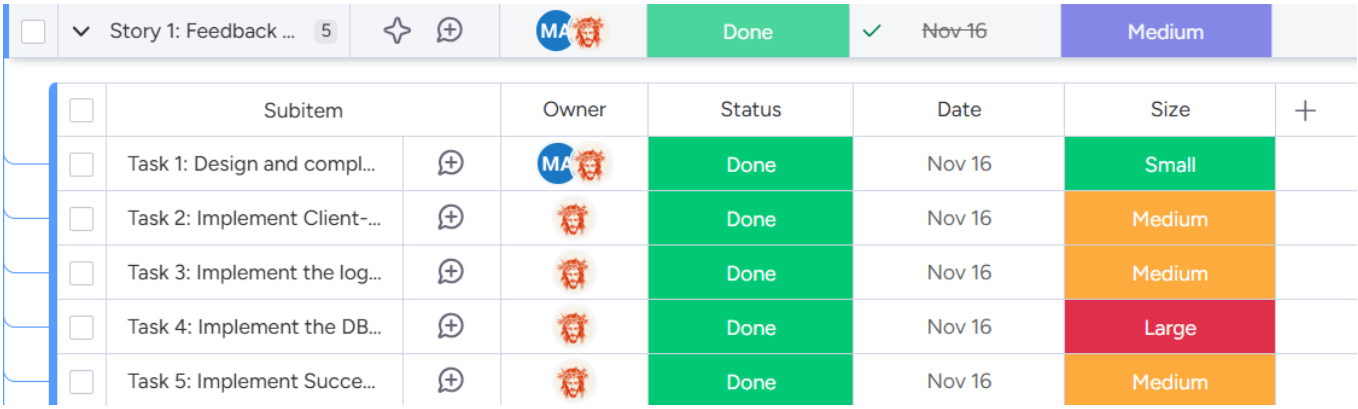
25. Update the Sprint dashboard, showing Sprint 4 with closed tasks and tasks you did not complete.
26. Dashboard, should clearly show task, owner, status, start date, end date, **size and priority.**
27. Must use a tool such as Trello or Monday. Word, Excel, … etc. will not be accepted. **(We are using Monday for scrum dashboard)**
28. Please use Scrum and not Kanban. **Scrum dashboard link: https://ma921833s-team.monday.com/**
29. Take a screenshot showing clearly the stories and breakdown of tasks with all details for sprint 4 only.
30. Must have a minimum of 4 stories and 5 tasks per story.

Picture of Dashboard

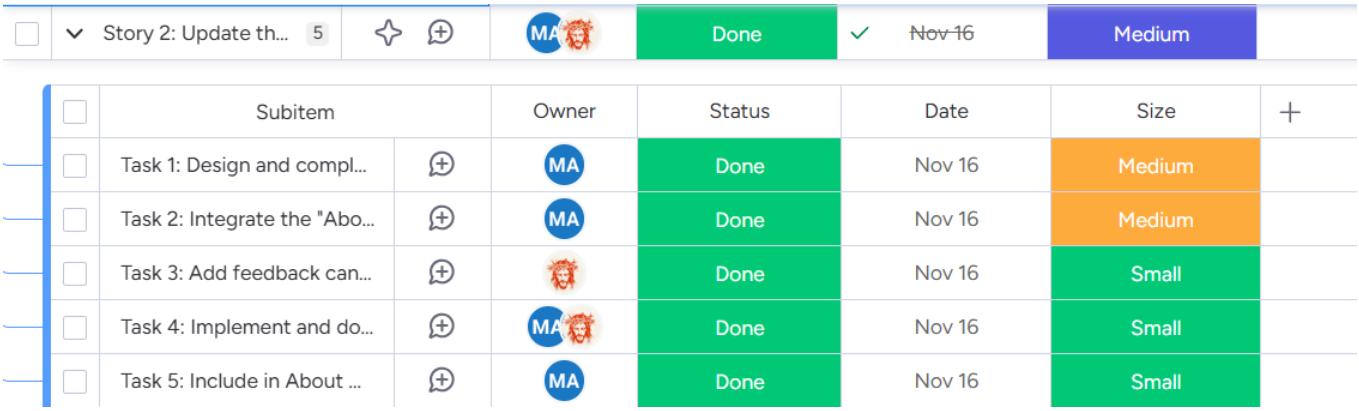
Epic 1:Functionality of the sensors and unit tests



Epic 2: Integration of the feedback and about us Functionality and Handling



Story 1: Feedback ... 5		Done	Nov 16	Medium
Subitem	Owner	Status	Date	Size
Task 1: Design and compl...	MA	Done	Nov 16	Small
Task 2: Implement Client-...		Done	Nov 16	Medium
Task 3: Implement the log...		Done	Nov 16	Medium
Task 4: Implement the DB...		Done	Nov 16	Large
Task 5: Implement Succe...		Done	Nov 16	Medium



Story 2: Update th... 5		Done	Nov 16	Medium
Subitem	Owner	Status	Date	Size
Task 1: Design and compl...	MA	Done	Nov 16	Medium
Task 2: Integrate the "Abo...	MA	Done	Nov 16	Medium
Task 3: Add feedback can...		Done	Nov 16	Small
Task 4: Implement and do...	MA	Done	Nov 16	Small
Task 5: Include in About ...	MA	Done	Nov 16	Small

31. **Take screenshots showing the team discussion, topics discussed, team members. Provide screenshots for three different dates, i.e. WhatsApp, Discord, …etc.**

32. **Use a tool to record your Sprint Retrospective (i.e.**
https://lucidspark.com/landing/create/online-sticky-notes

https://miro.com/

1. **Who missed the meeting, marks will be deducted for missing the retro.**

1. Answer the questions below, a minimum of 3 for each of the following:

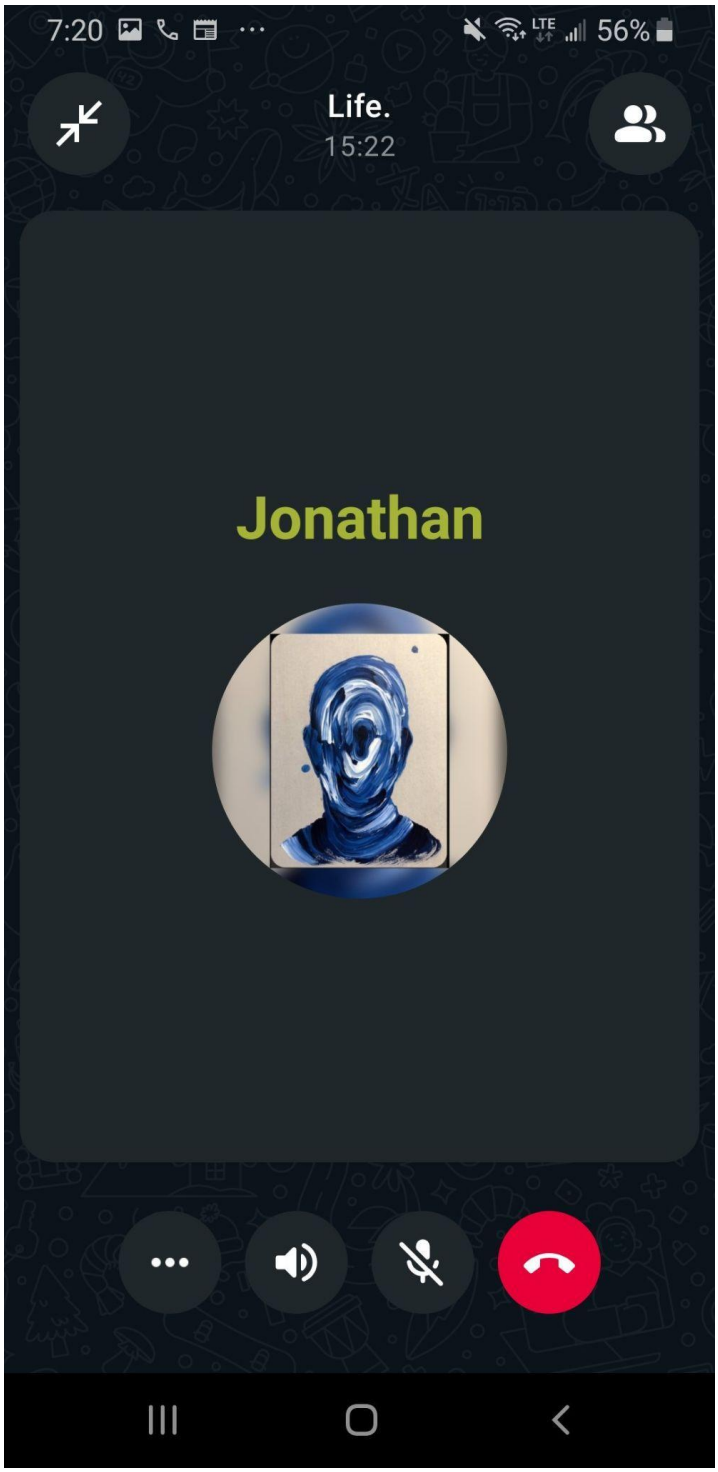
1. **Start doing.**
2. **Stop doing.**
3. **Continue doing.**

2. **Take a screenshot.**
3. **Must use online tool for the Retrospective. No mark will be given if no tool used.**

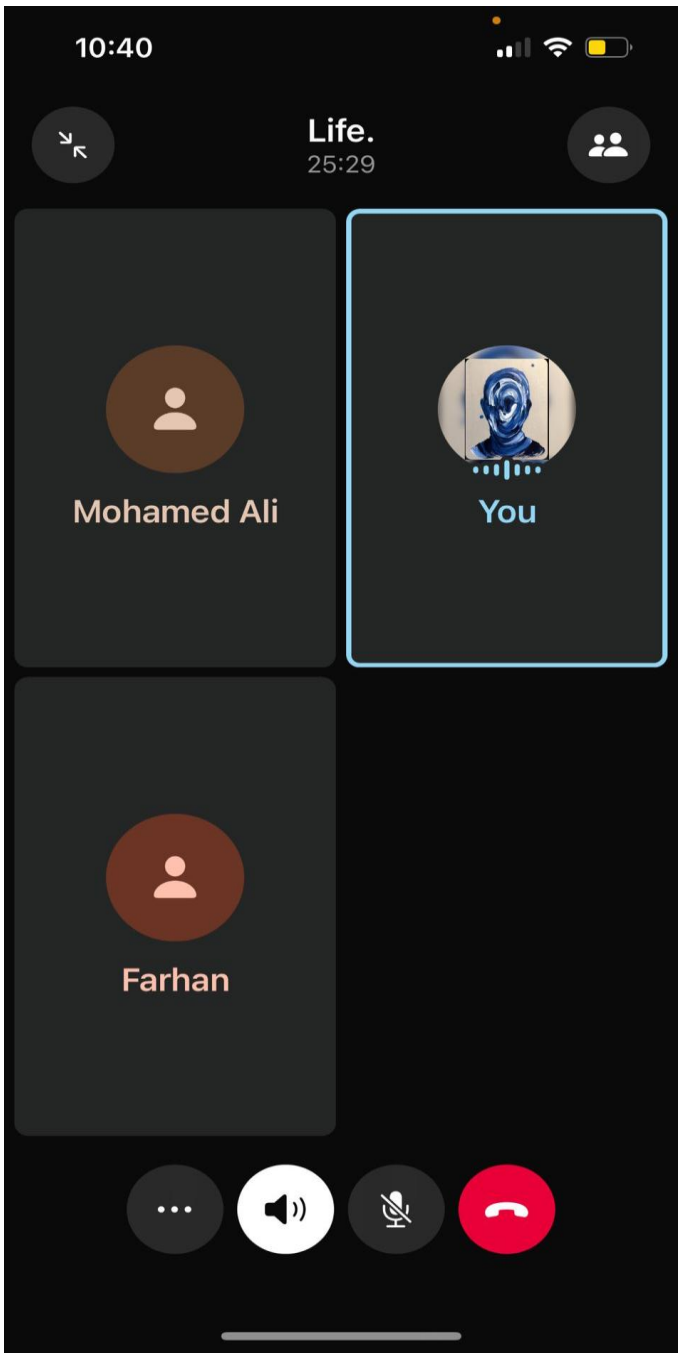
Miro links for the retrospective

https://miro.com/app/board/uXjVJsqFu7k=/

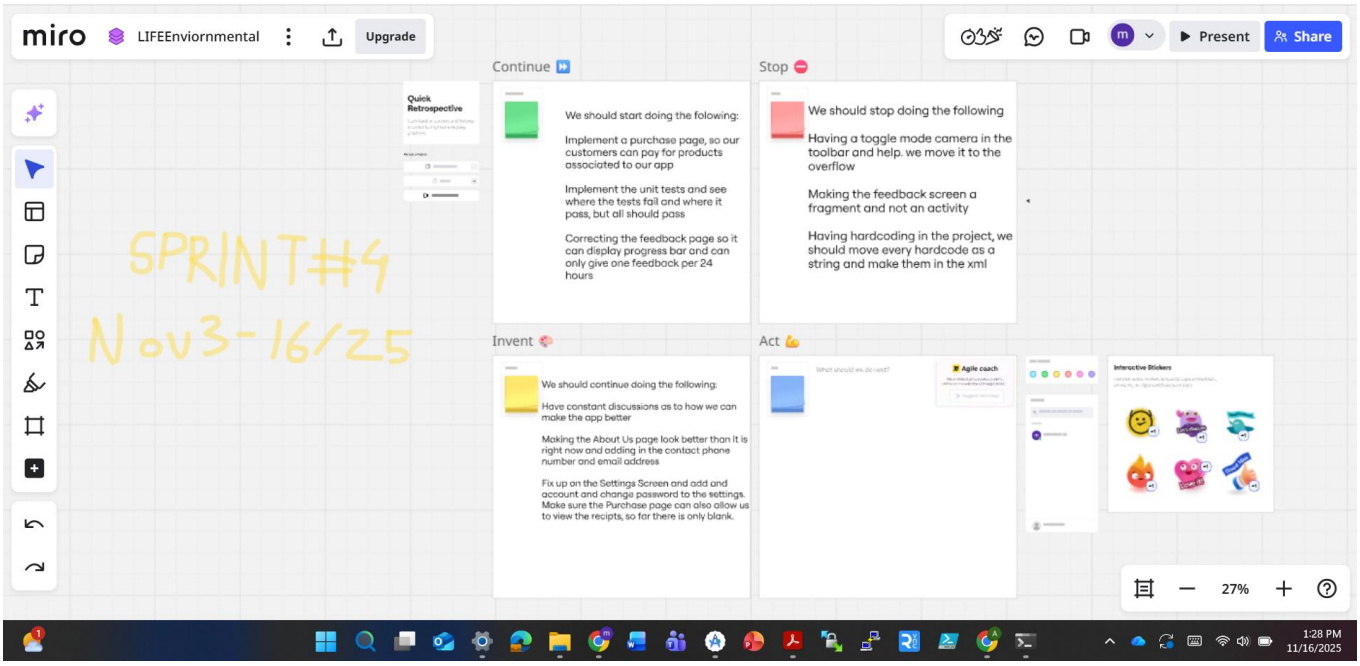
Meeting Nov 10, 2025


7:20
56%
Life.
15:22
Jonathan

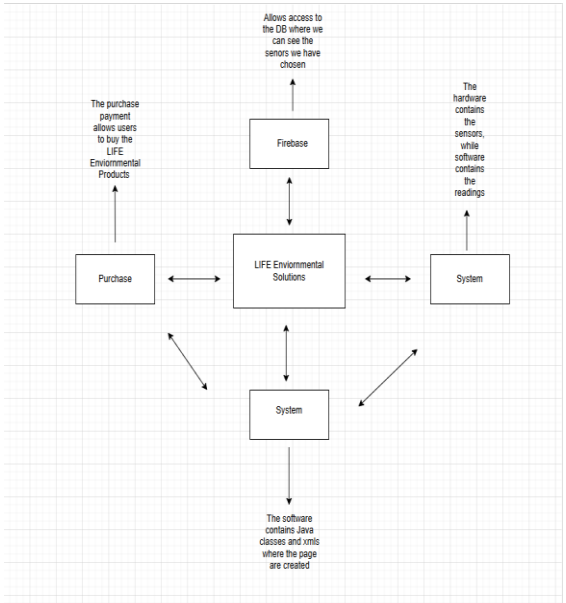
Nov 16, 2025



Miro Picture Retrospective



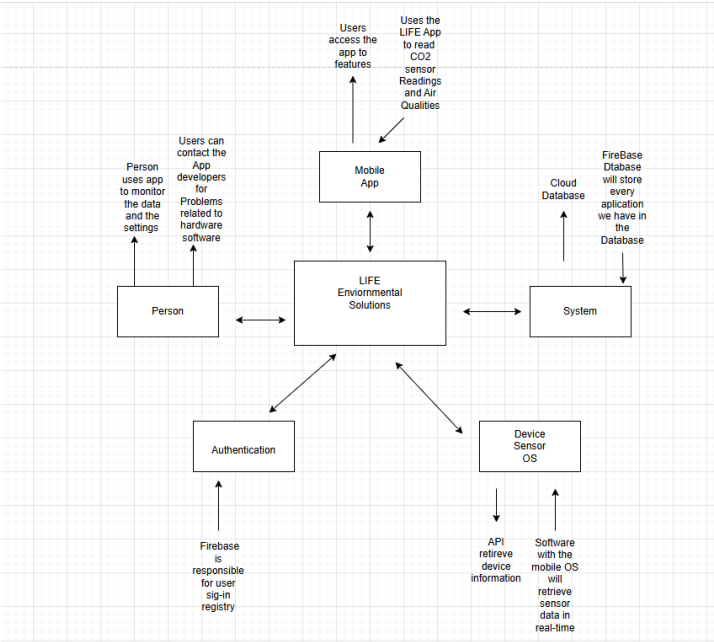
33. **Using C4 Model, show "System Context Diagram". Use a tool to draw your diagram, hand drawing will not be accepted, i.e. you can use. Screenshot Below https://app.diagrams.net/ System diagram link: https://app.diagrams.net/#G1Ba5K0ihH35-WLcqfG1nS3ZUF3-2UO6R7#%7B%22pageId%22%3A%22FQo-CGrpc4sa9dBLHK2_%22%7D**



34. **Using C4 Model, show "Container Diagram". Use a tool to draw your diagram, hand drawing will not be accepted, i.e. you can use https://app.diagrams.net/**

Context diagram link:
ttps://app.diagrams.net/#G1PM7Gx5k3j4tF5TOiS4z7qG415-swh_ej#%7B%22pageId%22%3A%22V9CZpbKbTyqKi0rJ4xxu%22%7D

Screenshot



35. Document two different design patterns used in the code. Copy the code you used, and add your explanation. Use the ones covered in the class.

Builder Patter in FeedbackPage.java

```
private void submitFeedback() {
  // Check if the user has submitted feedback in the last 24 hours
  SharedPreferences prefs = requireActivity().getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE);
  long lastSubmissionTime = prefs.getLong(LAST_SUBMISSION_TIMESTAMP, 0);
  long currentTime = System.currentTimeMillis();

  if (currentTime - lastSubmissionTime < TWENTY_FOUR_HOURS_MILLIS) {
    // It's been less than 24 hours
    new AlertDialog.Builder(requireContext())
        .setTitle("Submission Limit")
        .setIcon(R.drawable.logolife)
        .setMessage("You can only submit feedback once every 24 hours.")
```

```
            .setPositiveButton(android.R.string.ok, null)
            .show();
        Log.w(TAG, "Submission blocked: User tried to submit feedback before 24-hour
cooldown.");
        return;
    }
```

In the FeedbackPage.java, we have the builder design pattern for the AlertDialog. This is going to call upon chains stemming from the AlertDialog.Builder object, and this is the essence of the Builder pattern. This method is separating the construction logic from the final object's representation. It will call on different builds, like message and the title and will show the exact thing you want when you go into the feedback page when running the app.

Singleton Pattern in DashBoardFragment.java

```
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);

    // Initialize views and Firebase Auth
    userGreeting = view.findViewById(R.id.usergreeting); // Make sure this ID exists in
fragment_dash_board.xml
    mAuth = FirebaseAuth.getInstance();

    // Load the user's information to set the greeting
    loadGreeting(new FirebaseUserDataProvider());
}
```

In DashBoardFragment.java, we have the singleton design pattern. In the Override method for the onViewCreated, the static factory method FirebaseAuth.getInstance() is designed for the access point for the single, shared instance of the FirebaseAuth object. This is the most important implementation of the Singleton pattern used by system-wide resource managers like Firebase Database**.**

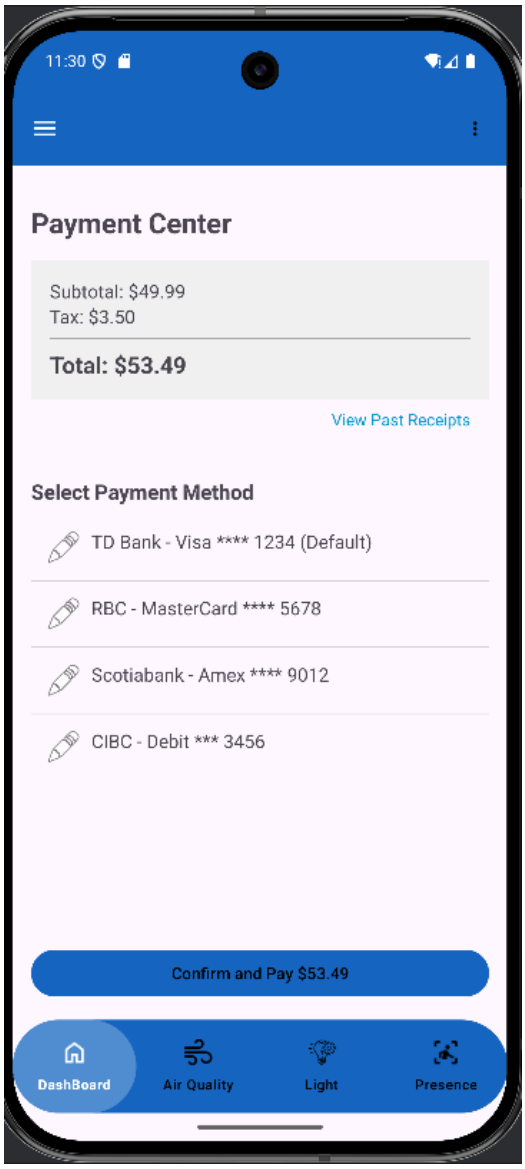
36. Your code should take Design Principles and Design Patterns into consideration, the ones covered in the class.

37. Coding work progress since deliverable 3. What additional features/functionality added since deliverable 3.

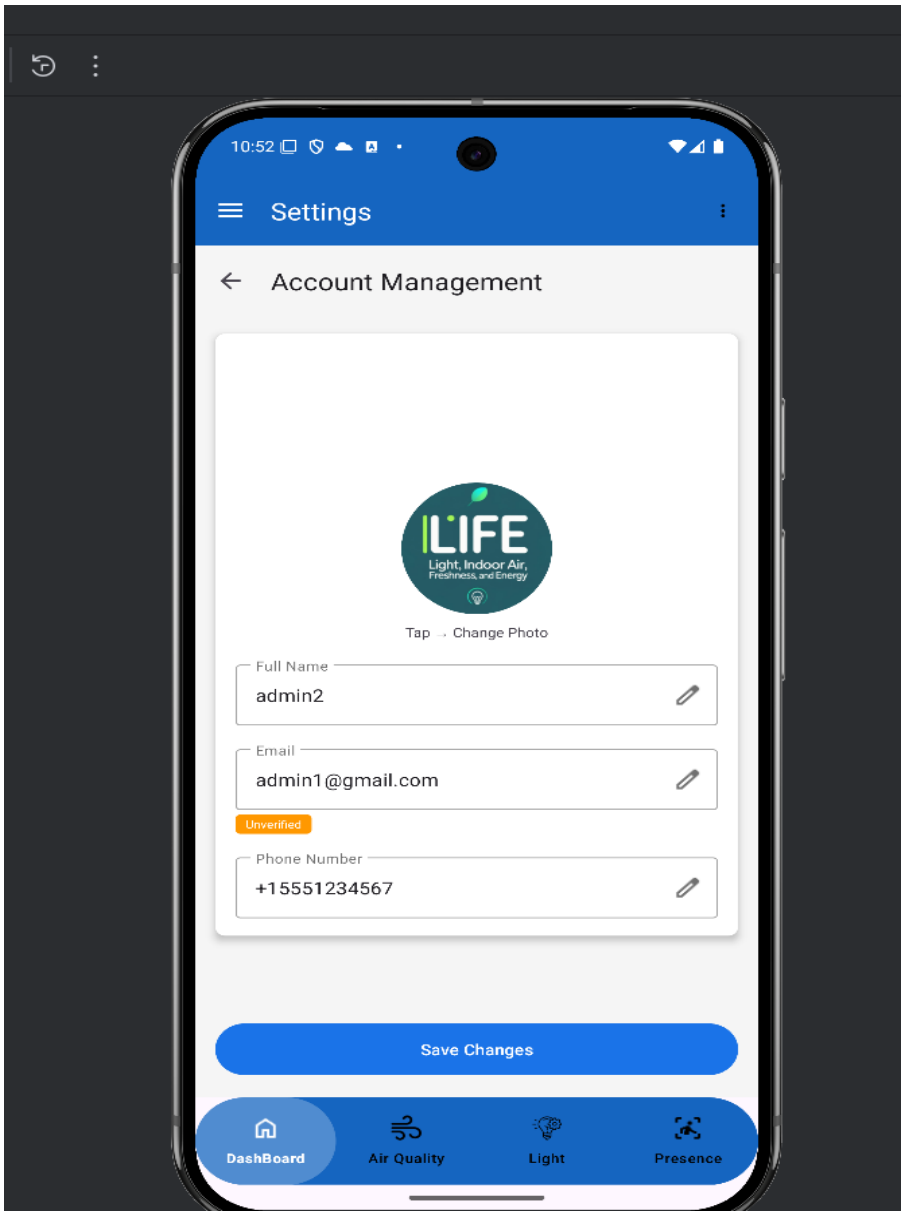
Additional features that have been added in since deliverable 3 is a purchase page. We first add the purchase to the navigation drawer, then we make the UI for it, and this is where we will add in payment methods. The payment methods in the UI is by using your debit or credit cards, as cash isn't accepted. Also there is the total amount, and in the payment center, you can use the view receipts. This is where you view your past receipts and can see what you exactly paid for and which product you ordered. In Java, we make

the methods that will call and take on the certain needs of the purchase page and the implementations that allow us to access the screen.
Also users can edit their details such as name email and phone number from the account management page

Picture showing the purchases page

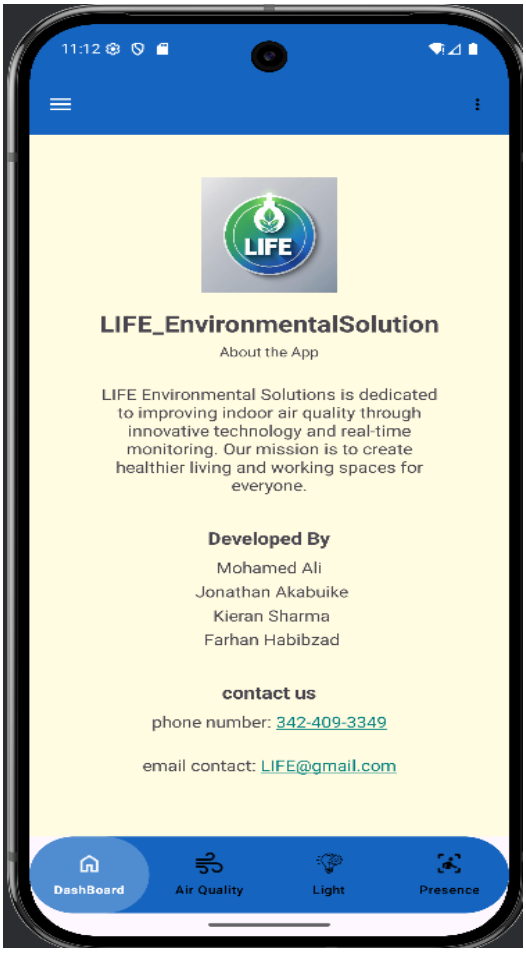


Screenshot showing the account management page



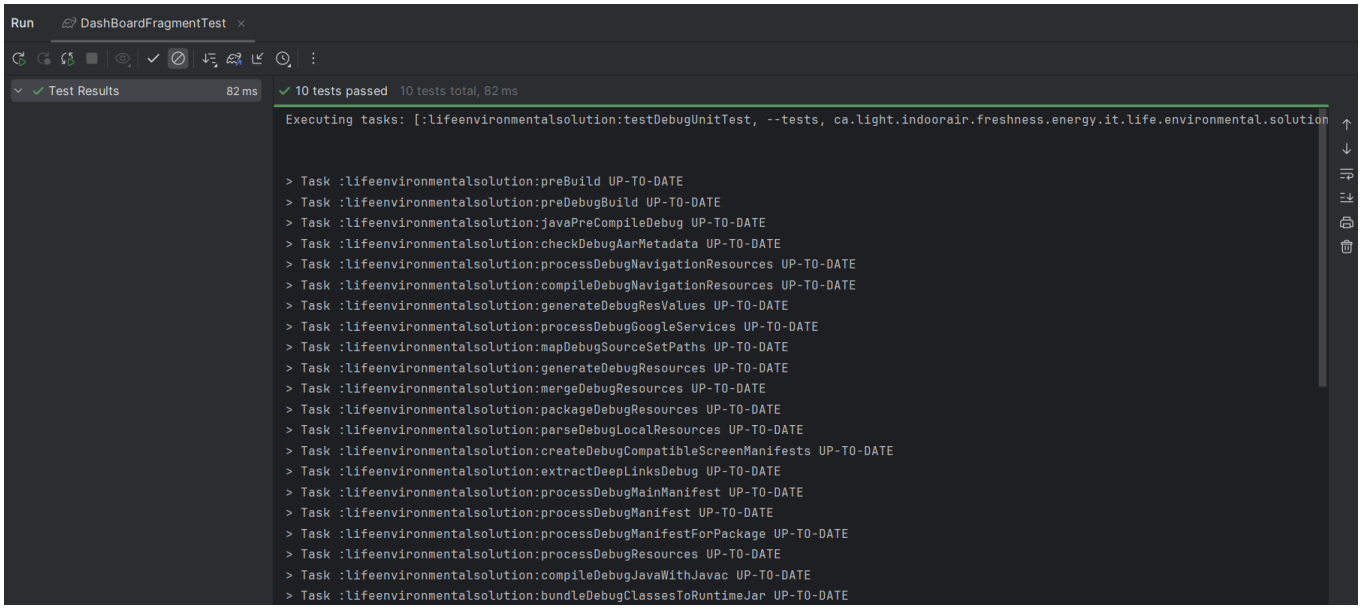
We have also added in the About us page. The about us page is an area where the users can read about us and what our product stands for. We want them to know us and if they have questions or concerns, then they can email us or call us and we will answer them

Picture showing the About Us page

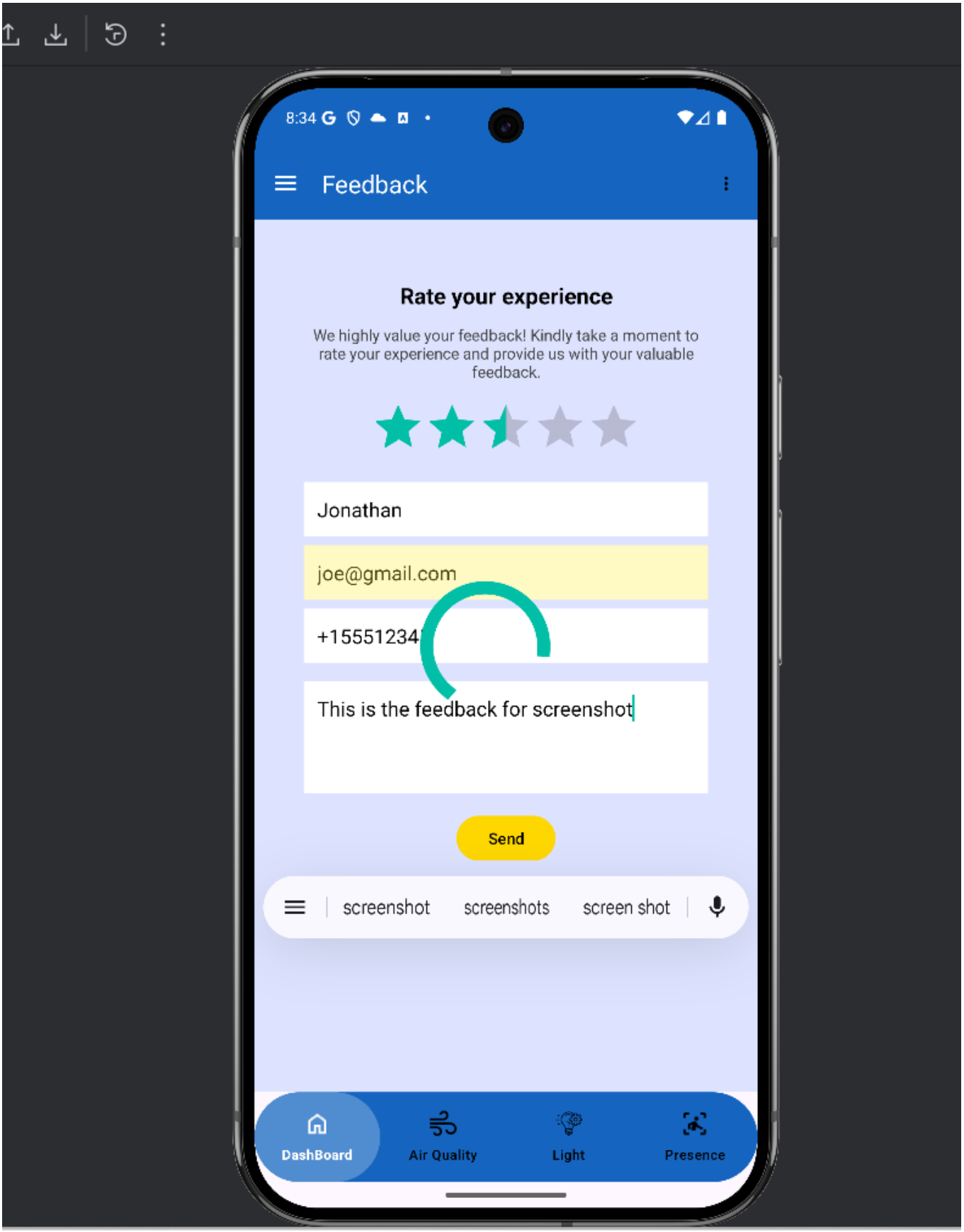


38. Write unit test cases. Select one of your Java classes and write a minimum of 10 test cases.
39. Demonstrate the use of assertEqual, assertTrue, assertFalse, and asssertNotNull in your testing.
40. All test cases must pass.
41. **Take a screenshot showing all your test cases are passing.**

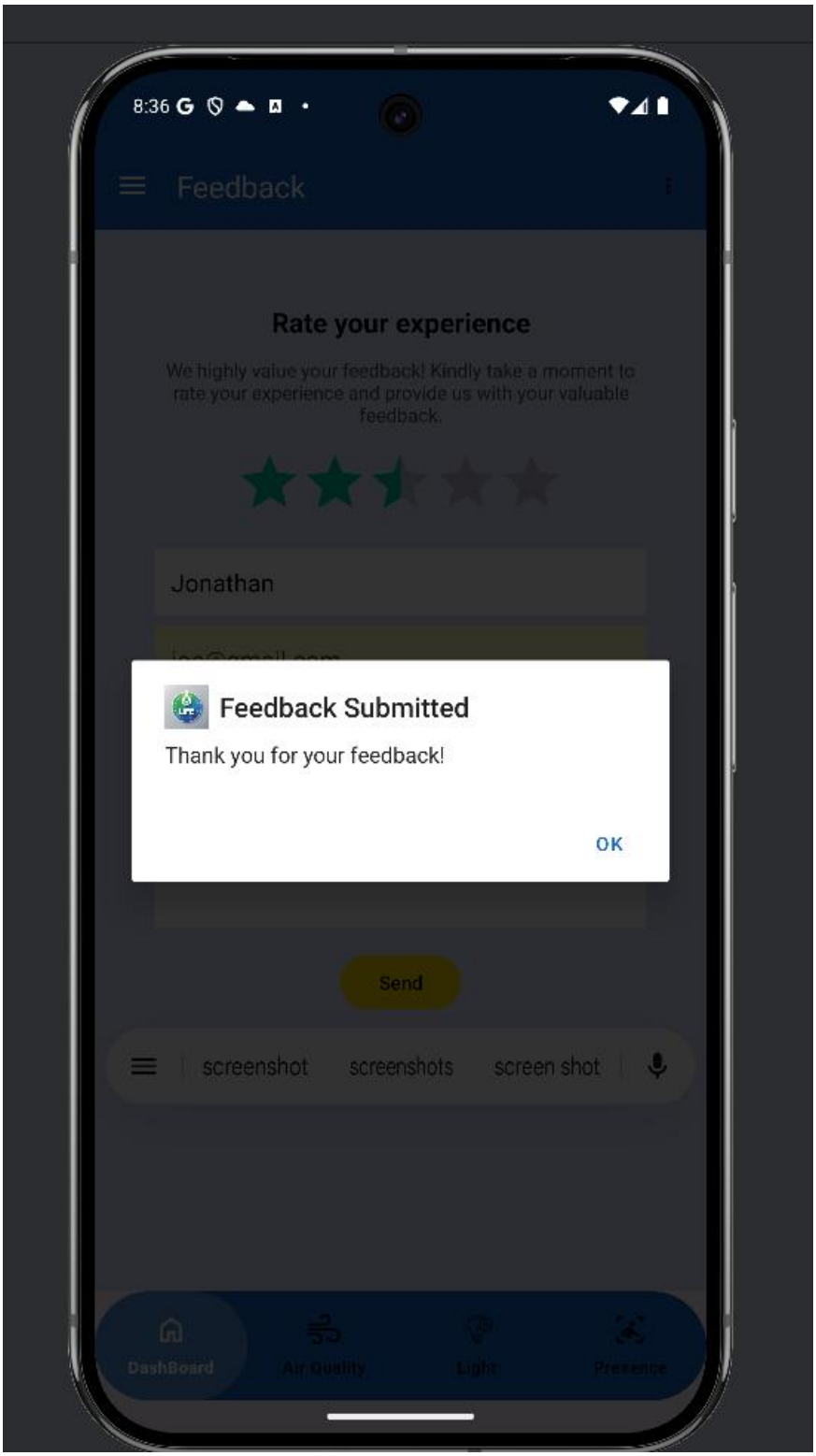
All unit tests are passing. AssertTrue, assertFalse, asserEquals and AssertNotNull are included



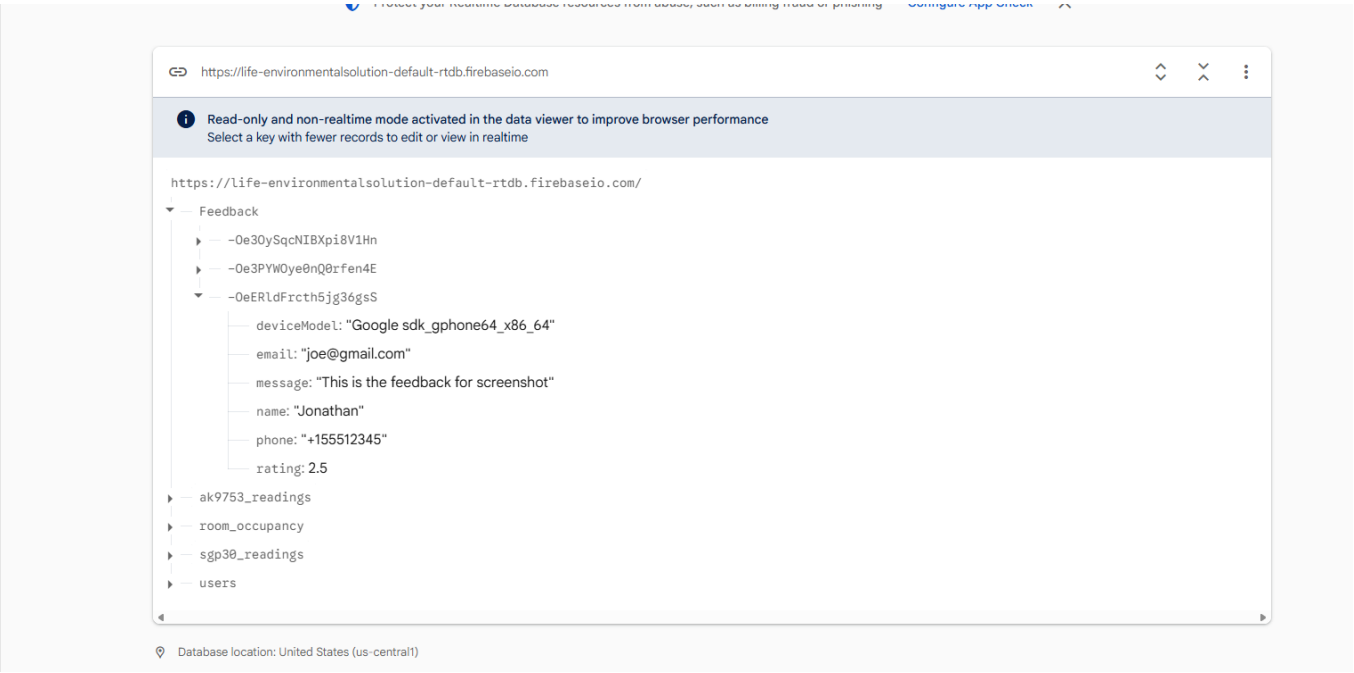
42. Functionality on the Customer Feedback Screen below.
43. Display an error and don't submit if invalid input, i.e. invalid phone number, …etc.
44. Display progress bar while the info is getting submitted, implement some delay for 5 seconds.
45. Once you receive a confirmation from the DB, display an AlertDialog with OK confirming the form has been submitted.
46. Verify you are submitting device model and stored into DB. You extract device model programmatically (don't ask for device model in the form). https://www.tutorialspoint.com/how-to-check-android-phonemodel-programmatically
47. **Add into pdf file screenshot showing the progress bar while the form is geknng submiled.**


8:34
Feedback
Rate your experience
We highly value your feedback! Kindly take a moment to rate your experience and provide us with your valuable feedback.
Jonathan
joe@gmail.com
+15551234
This is the feedback for screenshot
Send
screenshot
screenshots
screen shot
DashBoard
Air Quality
Light
Presence

48. **Add into pdf file screenshot showing the AlertDialog once the form is submiled successfully.**



49. **Add into pdf file screenshot showing the data stored into the DB. Must have at least 3 different entries.**



50. Clear the user input once the form is submitted. Restrict user submission to once per 24 hrs.

51. Once the user submits the form successfully, gray out the submit button, and display a timer showing how many hours and minutes remaining when the user can submit another feedback.

52. **Describe in the pdf file on how you satisfied this requirement.**
Explanation:

To satisfy the requirement of restricting user feedback submissions to once every 24 hours and to provide clear visual feedback ,a multilayered system was implemented in **FeedBackPage.java**.
This system relies on **SharedPreferences** for persistence and a **CountDownTimer** for UI updates.

The implementation has **three main stages**:

1. Checking the Cooldown on Screen Load

When the feedback page opens, the checkSubmissionCooldown() method is in onViewCreated(). Its job is to determine if the user is still on cooldown.

- **Timestamp Retrieval**

- The method reads a SharedPreferences file named **FeedbackPrefs**.
- It retrieves the last submission time using the key **LAST_SUBMISSION_TIMESTAMP**.
- If no timestamp exists, the default value is **0**.

● **Time Calculation**

- It compares the current system time with the stored timestamp.

● **Conditional Logic**

- **If less than 24 hours have passed:**
  - The **Send** button is disabled.
  - A countdown is started using startTimer() with the remaining time.
- **If 24 hours or more have passed:**
  - The **Send** button is enabled.
  - The timer TextView (tvTimer) is hidden.

Java Example

```
// From checkSubmissionCooldown()
SharedPreferences prefs =
requireActivity().getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE);
long lastSubmissionTime = prefs.getLong(LAST_SUBMISSION_TIMESTAMP,
0);
long timeDifference = System.currentTimeMillis() - lastSubmissionTime;

if (timeDifference < TWENTY_FOUR_HOURS_MILLIS) {
   btnSend.setEnabled(false);
   long remainingTime = TWENTY_FOUR_HOURS_MILLIS - timeDifference;
   startTimer(remainingTime);
} else {
   btnSend.setEnabled(true);
   tvTimer.setVisibility(View.GONE);
}
```

2. Storing the Timestamp After a Successful Submission

A new cooldown period begins **only after** the feedback is successfully written to Firebase.

● Database Confirmation

- Timestamp storage occurs inside the Firebase onCompleteListener, **only if** task.isSuccessful().

● Saving the Timestamp

- Inside the success dialog's positive button handler:
    - The current time (System.currentTimeMillis()) is saved to SharedPreferences as **LAST_SUBMISSION_TIMESTAMP**.
    - This marks the start of the next 24-hour cooldown.

● Starting the Cooldown

- After saving the timestamp and clearing the form fields, the full countdown begins using:
    - startTimer(TWENTY_FOUR_HOURS_MILLIS);

Java Example

```
// From the .setPositiveButton listener in submitFeedback()
SharedPreferences.Editor editor = requireActivity()
   .getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
   .edit();

editor.putLong(LAST_SUBMISSION_TIMESTAMP,
System.currentTimeMillis());
editor.apply();

// ... form fields are cleared ...
startTimer(TWENTY_FOUR_HOURS_MILLIS);
```

3. Displaying the Live Countdown Timer

The startTimer() method handles the real-time countdown visible to the user.

● UI Updates

- The **Send** button is disabled.
- The countdown TextView is shown.

● CountDownTimer Functionality

- A CountDownTimer is initialized with the remaining cooldown duration.

onTick()

- Updates every second.
- Converts the remaining milliseconds into **HH:mm:ss** format.
- Displays it in tvTimer.

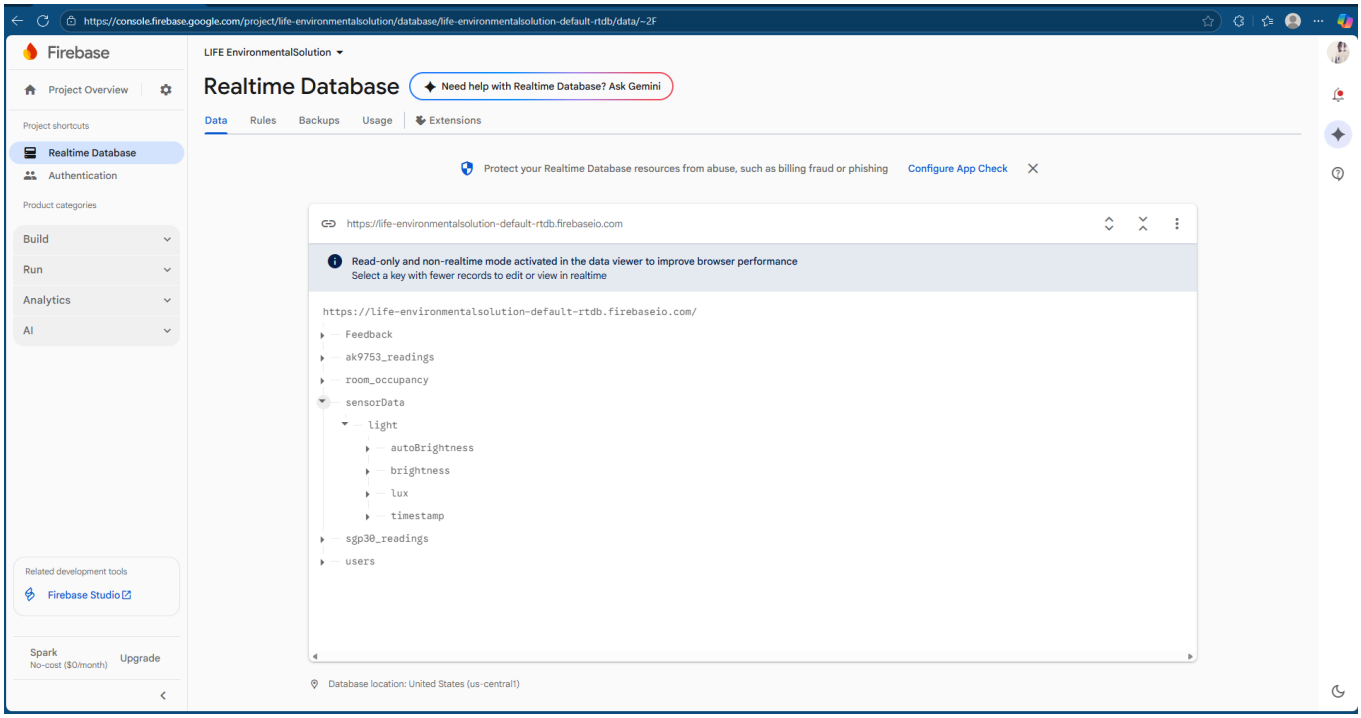
onFinish()

- Hides the timer.
- Re-enables the **Send** button automatically.

● Lifecycle Management

- The timer is cancelled in onDestroyView() to prevent memory leaks.

53. Take screenshot showing all sensor data fetched and updated from the DB.



54. Must implement at least two features of your application, they are related to the core functionality. Describe what main functionality added.

Over this sprint, two major features have been added that form the core functionality of the application: Real-time Environmental Monitoring with Cloud Integration and Synchronized State Management Across App Components.

1. Real-time Environmental Monitoring with Cloud Integration

Main Functionality:
The app is now fully integrated with Firebase Realtime Database to monitor and display live environmental data. This enables real-time IoT functionality, giving users instant updates as sensor values change.

Key Implementations:

Air Quality, Temperature, Comfort Level:
The dashboard listens to the room_occupancy path in Firebase and updates the main weather card immediately when new data arrives.

Light Intensity:

The LightFragment both writes simulated LUX values to Firebase and reads them back, keeping the displayed light level synchronized with cloud data.

Presence Detection:
The PresenceFragment listens to real-time updates from the same Firebase path to show whether a room is “Occupied” or “Empty.”

2. Synchronized State Management Across App Components

Main Functionality:
A unified state management system ensures that user settings and feature states remain consistent across all screens. Any change made in one fragment is immediately reflected on the dashboard.

Key Implementations:

Shared Preferences as Single Source of Truth:
Feature states are stored in SharedPreferences, and fragments register listeners to detect changes.

Smart Light:
The “Auto Brightness” toggle in LightFragment saves its state to SharedPreferences. The dashboard listens for updates and instantly changes the Smart Light icon and status.

Presence Sensor & Air Quality:
When users enable or disable presence detection or when air quality updates, the changes are saved to SharedPreferences. The dashboard receives these changes and updates its UI accordingly**.**

55. **Describe the main functionality added in this sprint.:**

During this sprint, several key features were implemented to enhance user interaction, account management, and system reliability. The primary additions include:

1. Purchase Screen Integration
   A fully functional Purchase screen was added, allowing users to purchase items directly within the app. Users can now view all their past receipts through a lifetime receipt history feature, improving transparency and record-keeping.

2. Enhanced Feedback System
   The Feedback screen was upgraded to support real-time data submission to the database. A submission-cooldown mechanism was implemented, restricting users to one feedback entry every 24 hours. This feature ensures controlled usage while providing users with a

clear countdown timer indicating when they can submit again.

3. Updated Settings and Account Management
   The Settings screen was redesigned to include a dedicated Account section. This allows users to create their own accounts and manage their information, establishing the foundation for personalized user experiences across the app.

56. Create a subfolder called deliverable 4 under docs in the repo and add the pdf file.

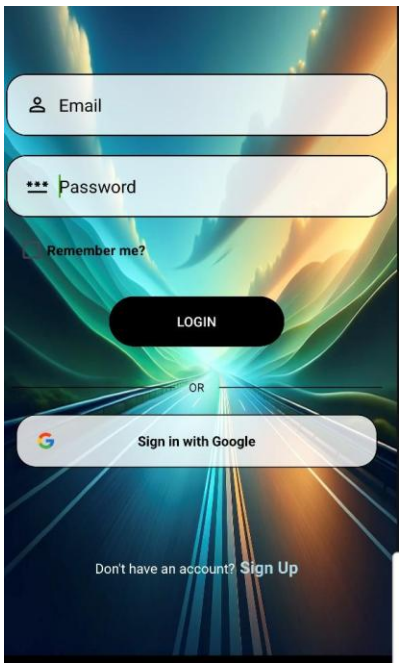
If document not in pdf, or not the proper name (marks will be deducted!)
Please save the PDF document, i.e.

TeamName_ProjectName_Deliverable4.pdf Deliverable 4:

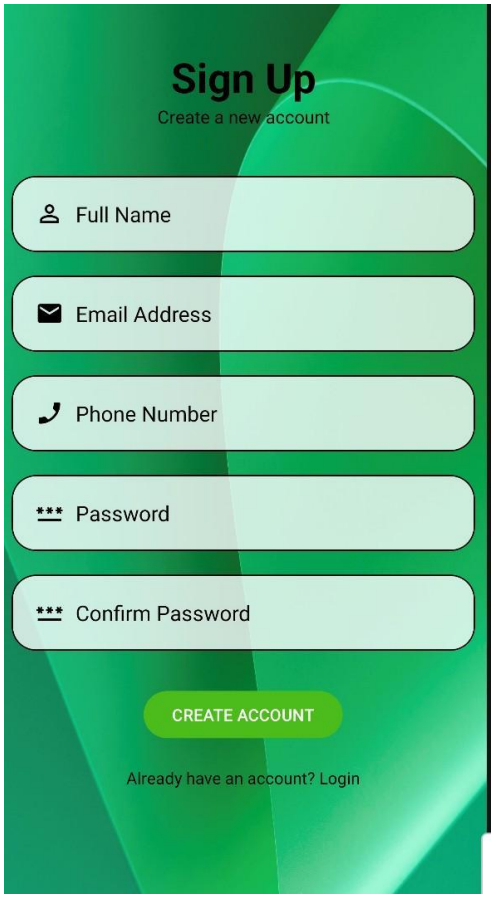
Android Studio Project pushed into in github:

1. Use NavigaTon Drawer with the combinaTon with another layout.
2. Must Complete the UI design. All the UX must be completed.
3. Should have a minimum of 10 Java/Kotlin classes.
4. Each member must have a minimum of 10 commits. Counted starting Nov. 3.
5. Must have a minimum of 7 screens (including the Splash, Login Registration, Feedback, Settings and Home Screen).
6. Splash, Login and Registration Screens must be activities and not Fragment. Users access these screens prior to accessing the main application. All other screens could be fragments.
7. Once a user login, they land on the Home Screen.
8. Must dedicate a minimum of 2 screens for the main app functionality (excluding Splash, Login, Registration, Settings, Feedback, Account, Help, …etc.).
9. Expect continuous commits for at least 8 days up to the delivery date. Marks deducted for not meeting this requirement.
10. You should have commits on at least 8 different days.
11. Use the proper name for the app name, don't use the word app.
12. Application name must be your project name.
13. Use the proper name for the package name, avoid the words example, test,…etc. in the package name, otherwise your app will be rejected.
14. Use the proper image name for the homescreen icons, replace the default android icon.
15. Minimum API 30 and target API 36/37.
16. Use two proper design patterns.
17. Complete the Top Bar menu implementation with all the functionality and UI. No further changes should be needed for the menu on the toolbar. Refer to Deliverable 2 for the requirements.
18. Implement read and write from the cloud DB. Data read from DB displayed on screen. Users update some data and update DB.

19. All sensor data fetched and updated from the DB.
20. Should allow for configuration setting and remember user selections. Store data into SharedPref and read it back, i.e. if the user selects Dark Theme, this option should be maintained when app restarts, device power off/on, …etc.
21. Complete the implementation of the settings screen.
22. Login logic, In addition to creating an account, allow the user to login using Gmail, without the need to create an account on your app.



23. Login logic in your application, must have a checkbox “Remember Me”. Implement the functionality, so user does not have to login every Tme access the app.
24. Login logic in your applicaTon, Complete the implementaTon of the registraTon and login screens.
25. Registration Screen, verify registration screen has the following fields: Name, phone number, email, password and confirm password. See how should have a link in the login screen to the Signup screen.
26. Validate user input on registration screen, i.e. valid phone number, …etc. Must validate all user input, in addition restrict user input, i.e. phone number field should be numbers only,…etc.



hbps://www.behance.net/gallery/75101999/Login-Screen-UI-UX

27. Remove username (userId) field. Email should be unique and used to identify users.
28. Login Screen:
    1. Login screen must not be part of the tabs.
    2. Must follow the credentials format below, build functionality to validate format on the Android App. Also verify credentials correct from the DB on the cloud.
        1. Username should be a valid email.
        2. Password validation:
            A. Password should be at least 6 in length.
            B. Should include alphabetic and numbers.
            C. Should include at least one special char.
            D. Include at least one upper case letter.
            E. Include at least one digit.
    3. Use Regex to validate credentials.
29. Registration screen:
    1. Full name.

2. Email, used as username. Must be valid email format.
3. Remove username (userId) field. Email should be unique and used to identify users.
4. Password, please refer to login section for the password format.
5. Confirm Password field.
6. Phone number must be valid 10 digits phone format.
7. Use hint attribute to help user in the registration.

30. Feedback Customer relationship (Business Model Canvas): In addition to existing screens; design and implement a new "Customer Feedback Screen". https://www.pinterest.ca/masoodux/review-pageinspiration/

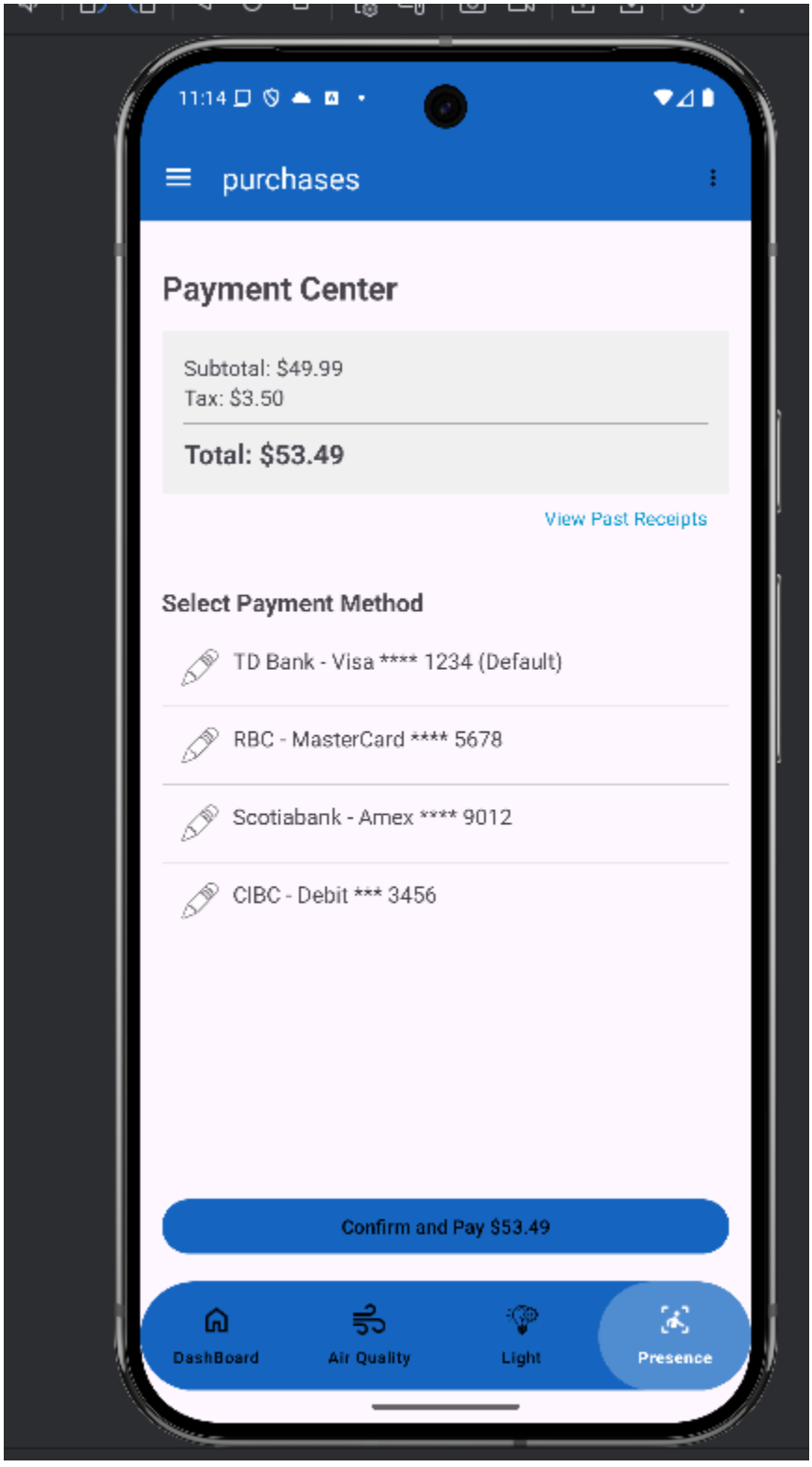
1. Use emoji or stars for the feedback.
2. Include, name, phone number, email and comment. Use proper formats for the entries, and the number of stars selected.
3. In addition to above read device model programmatically, and pass to the DB. Device model will not be shown on the feedback screen, however stored on the DB.

https://www.tutorialspoint.com/how-to-check-android-phone-model-programmatically-using-kotlin

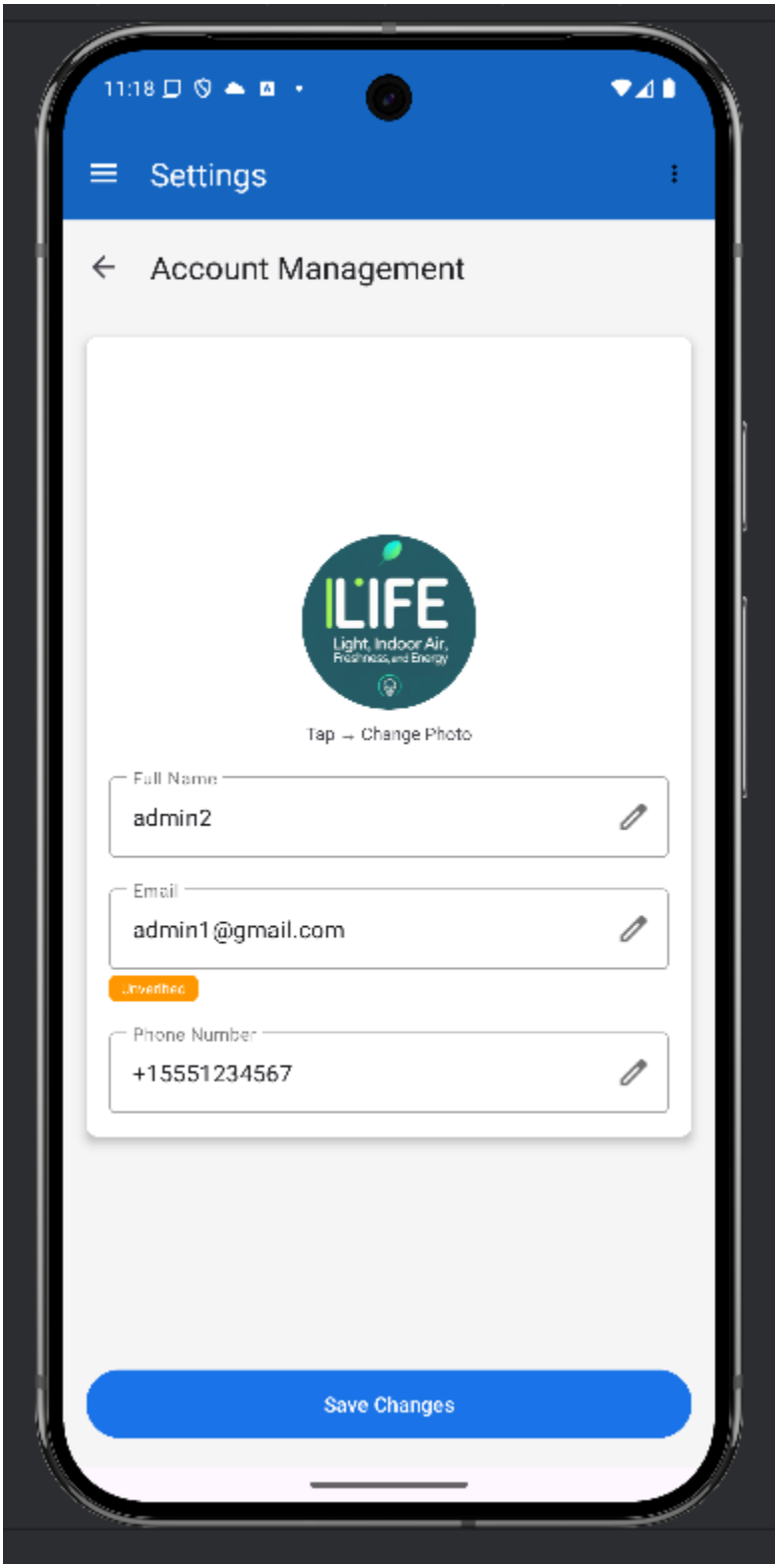
4. Store data into DB on cloud for assessment.
5. Refer to requirements listed above in the pdf section about Customer Feedback screen.

31. The following screens, the design and functionality must be complete: splash screen, login screen, registration screen feedback screen and **settings screen.**
32. Must complete the UI design and incorporate all the changes requested. No additional UI should be needed for the subsequent submissions.
33. Allow the user to log out from your app. If they access it again, they need to login.
34. Must implement at least two features of your application, they are related to the core functionality.
35. Document and screenshots of the two features of your app that you implemented in this sprint. (purchase screen and the account management page)

Picture of the Purchase screen



Screenshot of the Account management



36. Plus deliverable 1 + 2 + 3.
37. The app must not have compilation errors.
38. The app must not crash.
39. The a will be tested on different simulators and devices.
40. Must not have an option on the Setting screen to switch Language. Language is set at the device level and not at the app level.

---------- Deliverable 3:

Refer to deliverable 3.